Neologik Customer Onboarding Scripts

Version: v5.0.0

Automated PowerShell scripts for onboarding customers to the Neologik Platform on Azure.

Overview

This repository contains a robust PowerShell script that automates the complete Azure environment setup for Neologik customers, including:

- PowerShell version validation and automatic updates
- Azure module dependency management
- Resource provider registration with retry logic
- Azure authentication with re-authentication option
- Permission validation (Subscription Owner and Global Administrator)
- Input validation for all configuration parameters
- Guest user invitations (optional - can be skipped)
- Subscription and resource group setup
- Security group creation and configuration
- App Registration (Service Principal) creation with secure secret storage, or reuse of a service principal you have created yourself
- 'Neologik Platform Sign-in' app registration for NCE and App portal login
- Key Vault setup with RBAC authorization
- Storage Account for certificate storage with Entra ID authentication
- Managed Identity setup with role assignments and retry logic
- Role-based access control (RBAC) assignments with retry logic for replication delays
- Checkpoint/resume support - an interrupted run can be resumed without repeating completed steps
- Comprehensive logging and error handling
- Configuration export with script version tracking

What's New in v5

- **No bots at onboarding**: agents and their channels are created after deployment in the Neologik Platform admin tool (NCE). The bot count / agent name prompts are gone; `botCount` / `bots` keys in older answer files are ignored.
- **Platform naming convention**: resource names are driven by two naming tokens - the solution name and solution short name - and the environment type joins the organisation code in every name (see Resource Naming Convention below).
- **Client naming conventions**: every script-created resource name can be overridden from the answer file, so a client-mandated naming convention can be applied without editing the script.
- **Client-provided service principal**: you may create the deployment service principal yourself and give the script its object ID; the script then assigns the required roles to it instead of creating one.
- **Skippable guest invitations**: `-SkipGuestInvitation` (or the answer file) skips inviting Neologik staff, for clients who provision accounts in their own tenant. Declining invitations offers a follow-up:

add existing pre-created accounts to the security groups (from the guest file or typed addresses) with no invitation sent; non-existent addresses are reported and skipped without stopping the run.
- **Platform Sign-in app**: the script creates the 'Neologik Platform Sign-in' app registration and assigns the NCE role groups to it, so NCE login works without a separate manual step.
- **Non-interactive (CI) mode**: `-NonInteractive` runs the script unattended from a fully-populated answer file.
- **Entra Agent ID permissions**: the deployment service principal is granted the granular Microsoft Entra Agent ID Graph roles needed to provision agent identities, instead of any broad directory role.

Prerequisites

System Requirements

- **Operating System**: Windows 11 (or Windows 10 with latest updates)
- **PowerShell**: 7.4 or higher (script will offer to install/update if needed)
- **Administrator Access**: Required for module installation and system updates
- **NeologikGuestUsers.txt**: Guest user email list file (ships in the onboarding package; keep it in the same folder as the script). If it is missing, interactive runs offer to locate it or continue without guest invitations.

Azure Requirements

- **Azure Tenant**: Active Microsoft Entra ID tenant
- **Subscription**: Azure subscription with appropriate billing setup
- **User Permissions**:
  - **Owner** role at the subscription level (required for RBAC assignments)
  - **Global Administrator** or **Privileged Role Administrator** role in Entra ID - required to grant the scoped Microsoft Graph app-role assignments to the service principal and to create the Platform Sign-in app group assignments
  - Note: "Contributor" or "Co-Administrator" roles are insufficient for role assignments

Network Requirements

- Internet connectivity for:
  - Azure portal access
  - PowerShell module downloads from PSGallery
  - Guest user invitation emails (when guest invitations are enabled)

Installation

Quick Start

1. **Clone or download this repository:**

```
git clone https://github.com/Neologik-Customer/neo-client-onboarding.git
cd neo-client-onboarding
```

2. **Open PowerShell 7 as Administrator:**

   Right-click PowerShell 7 and select "Run as Administrator"

3. **Run the script:**

   ```
   .\Install-NeologikEnvironment.ps1
   ```

   The script will prompt you for all configuration values with sensible defaults. Simply press Enter to accept defaults, or type your own values.

Command-Line Parameters (Optional)

You can also provide parameters directly to skip some prompts:

```
.\Install-NeologikEnvironment.ps1 -OrganizationCode "ABC"
```

Or specify several parameters:

```
.\Install-NeologikEnvironment.ps1 `
    -OrganizationCode "XYZ" `
    -EnvironmentType "prd" `
    -AzureRegion "ukwest" `
    -SubscriptionName "Neologik-Production-01"
```

Script Parameters

Parameter	Type	Default	Description
`OrganizationCode`	String	(prompted)	3-character organization code for resource naming. Exactly 3 alphanumeric characters, lowercased automatically
`EnvironmentType`	String	`"dev"`	Environment type: `dev` or `prd`. Joins the organisation code in every computed resource name (e.g. `abcprd01`)
`AzureRegion`	String	`"uksouth"`	Azure region for resource deployment

Parameter	Type	Default	Description
`TenantId`	String	(current)	Azure Tenant ID (optional, uses current context)
`SubscriptionName`	String	`"Neologik-Development-01"`	Name of the Azure subscription to use
`KeyVaultName`	String	(auto: `kv<org><env><index>`)	Override the Key Vault name. 3-24 chars; start with a letter, end with a letter or digit; letters, digits and hyphens only
`SkipPowerShellUpdate`	Switch	`false`	Skip PowerShell version check and update
`SkipModuleInstall`	Switch	`false`	Skip Azure module installation check
`SkipGuestInvitation`	Switch	`false`	Skip inviting Neologik guest users. Use when the client provisions its own accounts in its own tenant. Security groups are still created, seeded with the signed-in user only
`ExistingServicePrincipalObjectId`	String	(none)	Object ID (enterprise-application object ID, not the app/client ID) of a service principal you have already created. The script skips app/SP/secret creation and assigns the required roles and Graph permissions to it
`ExistingServicePrincipalSecret`	String	(none)	Client secret for the pre-existing service principal, stored in Key Vault for the deployment pipeline. Optional - if omitted,

Parameter	Type	Default	Description
			add the secret to Key Vault manually before the first deployment
`AnswerFile`	String	(auto-detect `inputs.json`)	Path to a JSON file of predefined inputs that pre-fill the interactive prompts
`NonInteractive`	Switch	`false`	CI/unattended mode; requires a fully-populated answer file (see below)
`AcceptTerms`	Switch	`false`	Non-interactive acknowledgement of the Terms of Use; required with `-NonInteractive`
`FreshRun`	Switch	`false`	Discard any leftover checkpoint and start clean without prompting

Answer File (`inputs.json`)

Every interactive prompt can be pre-filled from a JSON answer file - copy `inputs-example.json` to `inputs.json` next to the script (auto-detected) or pass a path with `-AnswerFile`. Values pre-fill the matching prompt: press Enter to accept, or type to override. The file also carries the provisioning decisions (`skipGuestInvitation` , `useExistingServicePrincipal` with its object ID and optional secret, `registerResourceProviders`) and the resource-name overrides described under Resource Naming Convention.

Non-Interactive (CI) Mode

`-NonInteractive` runs the script unattended, for automation such as GitHub Actions:

- A fully-populated answer file is required (`organizationCode` , `organizationName` , `fqdn`) and is verified before anything runs.
- The caller establishes the Azure and Microsoft Graph contexts **before** invoking the script; they are always reused (no re-auth prompt):

```
Connect-AzAccount -AccessToken $armToken -AccountId $upn -TenantId $tenantId -
SubscriptionId $subId
Connect-MgGraph -AccessToken (ConvertTo-SecureString $graphToken -AsPlainText
-Force)
```

```
pwsh -NonInteractive -File ./Install-NeologikEnvironment.ps1 -NonInteractive -
AcceptTerms -FreshRun `
    -AnswerFile ./inputs.json -SkipPowerShellUpdate -SkipModuleInstall -
SkipGuestInvitation `
    -SubscriptionName $subName -ExistingServicePrincipalSecret
$env:SP_CLIENT_SECRET
```

- Terms of Use must be acknowledged with `-AcceptTerms` (logged).
- The configuration summary is not confirmed interactively; warn-and-prompt checks (Global Administrator verification, subscription selection) fail hard instead of prompting.
- A leftover `NeologikCheckpoint.json` is an error unless `-FreshRun` discards it.
- Access tokens are short-lived (~60 minutes) and cannot be refreshed mid-run; the token user still needs Subscription Owner + Global Administrator.
- The upload of the configuration file to the Storage Account is skipped (data-plane call the static token session cannot authenticate) - collect the local files instead.
- Run under `pwsh -NonInteractive` so any unexpected prompt throws instead of hanging.

Usage Examples

Example 1: Interactive Setup (Recommended)

```
.\Install-NeologikEnvironment.ps1
```

The script prompts for every value with a default shown; press Enter to accept or type a new value.

Example 2: With Organization Code Parameter

```
.\Install-NeologikEnvironment.ps1 -OrganizationCode "ABC"
```

This sets the organization code to "ABC" and prompts for other values.

Example 3: Production Environment Setup

```
.\Install-NeologikEnvironment.ps1 `
    -OrganizationCode "XYZ" `
    -EnvironmentType "prd" `
    -AzureRegion "ukwest"
```

Example 4: Skip Updates (Already Configured System)

```
.\Install-NeologikEnvironment.ps1 `
    -OrganizationCode "ABC" `
```

```
    -SkipPowerShellUpdate `
    -SkipModuleInstall
```

Example 5: Use a Service Principal You Created Yourself

```
.\Install-NeologikEnvironment.ps1 `
    -OrganizationCode "ABC" `
    -ExistingServicePrincipalObjectId "00000000-0000-0000-0000-000000000000" `
    -ExistingServicePrincipalSecret $env:SP_CLIENT_SECRET
```

Supply the **enterprise-application (service principal) object ID**, not the app/client ID - the script resolves the client ID from it. The secret is optional; if omitted, add it to Key Vault manually before the first deployment (secret name `neologik-deployment-service-principle-secret`). Step-by-step creation instructions: CREATE-SP-GUIDE.md.

Example 6: Override the Key Vault Name

```
.\Install-NeologikEnvironment.ps1 `
    -OrganizationCode "ABC" `
    -KeyVaultName "kv-abc-shared"
```

By default the Key Vault is named `kv<org><env><index>` (e.g. `kvabcprd01`). Supply `-KeyVaultName` (or the `keyVaultName` answer-file key) to use a name of your choosing instead.

The chosen name is saved to the checkpoint, so if a run fails and you resume it, the same name is reused automatically - you do **not** need to pass `-KeyVaultName` again. If you *do* pass `-KeyVaultName` on a resume, the command-line value takes precedence over the saved one.

Interactive Configuration Flow

The script asks for, in order: Organization Name, Bot Hostname (FQDN), Organization Code, Environment Type, Azure Region, Environment Index, resource tagging (with optional custom tags), then the five resource names (Resource Group, Key Vault, Deployment Storage Account, Script Runner Managed Identity, SQL Managed Identity - each pre-filled with the computed standard name or your answer-file override), the budget cost-report email recipients, the resource-provider decision (register the full platform provider set now, or only the minimal set the script itself needs), the guest-invitation decision (accepting it lists the file's addresses and offers to add more, typed line-by-line; declining it offers a follow-up: add existing pre-created accounts to the security groups, from the guest file or typed addresses), and the service-principal decision (create one, or use one you already created). A configuration summary is shown for confirmation before anything is created.

Input Validation

The script includes comprehensive validation for all inputs:

- **Organization Name**: Required, cannot be empty
- **Bot Hostname (FQDN)**: Required, must be a valid fully-qualified domain name; any scheme/path is stripped and it is converted to lowercase
- **Organization Code**: Must be exactly 3 alphanumeric characters, automatically converted to lowercase
- **Environment Type**: Must be either 'dev' or 'prd', automatically converted to lowercase
- **Azure Region**: Must be a valid Azure region name (e.g., uksouth, eastus, westus), automatically converted to lowercase
- **Environment Index**: Must be a number between 01 and 99
- **Resource Group Name**: Azure naming rules (1-90 characters, alphanumerics, underscores, hyphens, periods, parentheses, cannot end with period)
- **Key Vault Name**: 3-24 characters, starts with a letter, ends with a letter or digit, letters/digits/hyphens only
- **Storage Account Name**: 3-24 characters, lowercase letters and digits only
- **Managed Identity Names**: 3-128 characters, letters/digits/hyphens/underscores, starting with a letter or digit

The script will keep prompting until valid input is provided, ensuring all resources are created with proper names.

Resource Naming Convention

Two naming tokens drive every computed name (the environment type joins the organisation code in both):

- **Solution name**: `neo-<org><env>-<region>-<index>` (e.g. `neo-abcprd-uks-01`)
- **Solution short name**: `<org><env><index>` (e.g. `abcprd01`)

Computed standard names (example: organization code `abc`, UK South, index `01`):

- **Resource Group**: `rg-<solution-name>` (e.g. `rg-neo-abcprd-uks-01`)
- **Key Vault**: `kv<short-name>` (e.g. `kvabcprd01`)
- **Deployment Storage Account**: `stneodeploy<org><env><region><index>` truncated to 24 characters (e.g. `stneodeployabcprduks01`)
- **Security Groups** (suffix = short name):
  - `Neologik User Group - abcprd01` (default users group - assigned to agent channels created in the NCE)
  - `Neologik NCE Contributors - abcprd01` (NCE contributor role; renamed in place from the pre-v5 `Neologik NCE User Group` where one exists)
  - `Neologik NCE Admins - abcprd01` (NCE admin role - Blueprints, Audit and the Admin page)
  - `Neologik Admin User Group - abcprd01` (Azure resource administration)
- **App Registrations**:
  - `Neologik GitHub Service Connection - abcprd01` (deployment service principal)
  - `Neologik Platform Sign-in - abcprd01` (NCE and App portal login)
- **Managed Identities**: `neologik-script-runner-service-connection-abcprd01`, `neologik-sql-managed-identity-abcprd01`

Client naming conventions: if your organization mandates its own naming convention, override any of the computed names from the answer file - `solutionName`, `solutionShortName` (the tokens; the short name also drives the group, app-registration and identity suffixes), `resourceGroupName`, `keyVaultName`, `storageAccountName`, `scriptRunnerIdentityName`, `sqlIdentityName`. Each is validated against the Azure naming rules above. Tell Neologik about your convention as well, so the platform deployment applies the same convention to the resources it creates inside the resource group.

What the Script Does

1. Pre-Flight Checks

- Validates PowerShell version (7.4+), offering to install/update if needed
- Checks for required Azure modules and installs missing ones
- Verifies Subscription Owner and Global Administrator roles

2. Azure Authentication

- Detects existing Azure login and prompts to re-authenticate or continue
- Connects to Azure and Microsoft Graph
- Handles the Azure mandatory-MFA claims challenge at sign-in where required

3. Guest User Management (optional)

Invites Neologik team members as guests based on email addresses in the `NeologikGuestUsers.txt` file.

File Format:

- One email address per line
- Blank lines are ignored
- Lines starting with `#` are treated as comments
- File must be in the same directory as the script

Users are only invited if no account with that email already exists in the tenant - accounts you have pre-created (B2B guests or full member accounts whose mail or UPN matches the listed address) are detected and used as-is. Skip invitations entirely with `-SkipGuestInvitation` or the `skipGuestInvitation` answer-file key.

Extending the invite list interactively: the invite prompt lists the file's addresses first; answering Y then offers "Do you want to add any more addresses to invite?" - typed one per line (blank line to finish, commas accepted within a line, duplicates removed). When the file is empty or missing, answering Y goes straight to typed entry, so the whole invite list can be built by hand. Unattended (`-NonInteractive`) runs use the file only.

Adding pre-created accounts without any invitations: when you decline invitations, a follow-up question asks "Add existing accounts to the security groups?". Answer Y to add accounts you have already created (guests or full members) to the groups with no invitation sent. The addresses come from `NeologikGuestUsers.txt`, or you can type them at the prompt - one address per line, Enter on an empty line to finish, with commas accepted within a line for pasting a ready-made list (duplicates are removed and the collected list is echoed back) - or supply them in the answer file (`addExistingAccountsToGroups` :

true, `existingAccountEmails` : array). Each address must exactly match the account's mail or user principal name; an address with no matching account is reported as not existing and skipped - the script always continues.

Important: accounts processed through this flow (invited or pre-existing) are added to **all four security groups**, including the Admin User Group, which carries Azure administration rights on the resource group. List only accounts that should hold Neologik admin/support access - your own end users are added to the specific group they need after deployment instead.

4. Resource Creation

Creates or validates:

- **Resource Group**: pre-created here so the deployment pipeline never needs subscription-level write access
- **Security Groups** (each seeded with the signed-in user plus any invited/pre-existing Neologik users):
  - `Neologik User Group - <short-name>` - default users group; agent channels created in the NCE reuse this group, or an administrator supplies another existing group at channel attach. The platform never creates groups itself
  - `Neologik NCE Contributors - <short-name>` and `Neologik NCE Admins - <short-name>` - NCE role groups, assigned to the Platform Sign-in application
  - `Neologik Admin User Group - <short-name>` - Azure resource administration

5. App Registration (Deployment Service Principal)

Creates (or, with a client-provided service principal, reuses):

- **App Registration**: `Neologik GitHub Service Connection - <short-name>`
- **Type**: Multi-tenant (AzureADMultipleOrgs)
- **Client Secret**: 365-day expiration, automatically stored in Key Vault
- **Roles** (assigned whether the SP was created here or supplied by you):
  - Reader (Subscription) - SKU/quota pre-flight reads only
  - Cost Management Contributor (Subscription) - cost-report configuration only
  - Contributor, Key Vault Certificates Officer, Key Vault Secrets Officer, Storage Blob Data Contributor, Azure Kubernetes Service RBAC Cluster Admin, Monitoring Contributor, Role Based Access Control Administrator (Resource Group)
  - Microsoft Graph app roles: Application.Read.All, Application.ReadWrite.OwnedBy, plus the Entra Agent ID roles (AgentIdentityBlueprint.Create, AgentIdentityBlueprint.AddRemoveCreds.All, AgentIdentityBlueprint.UpdateAuthProperties.All, AgentIdentityBlueprintPrincipal.Create, AgentIdentity.Create.All, AgentIdUser.ReadWrite.All)
- **Added to**: `Neologik Admin User Group - <short-name>`

6. Platform Sign-in App Registration

Creates the `Neologik Platform Sign-in - <short-name>` app registration (NCE and App portal login), stores its client secret in Key Vault, registers the deployment service principal as owner of both the application and its service principal (enterprise application) objects, and assigns the NCE Contributors and

NCE Admins groups to it so their members' tokens carry the groups claim the NCE authorizes from. The deployment pipeline later finds this app by its Graph uniqueName and reuses it.

7. Key Vault

- **Key Vault**: `kv<short-name>` (e.g. `kvabcprd01`), or the name you supplied
- **RBAC Authorization**: Enabled (no access policies)
- **Secrets Stored**: deployment service principal client secret (automatic), Platform Sign-in client secret (automatic), certificate PFX password (manual step)
- **Permissions**: Key Vault Secrets Officer assigned to the Neologik Admin User Group at resource group level

8. Resource Provider Registration

Registers the minimal Azure resource providers the script itself needs (Microsoft.Compute, Microsoft.KeyVault, Microsoft.ManagedIdentity, Microsoft.Resources, Microsoft.Storage), waiting for registration to complete with retry logic. A prompt (default No; pre-fillable with the `registerResourceProviders` answer-file key) offers to register the full platform provider set (around 30 providers) now instead. If you decline, run `Register-NeologikResourceProviders.ps1` from this repository before the platform deployment.

9. Storage Account

- **Storage Account**: `stneodeploy<org><env><region><index>` (or your override)
- **Authentication**: Microsoft Entra ID only (shared key access disabled)
- **Blob Containers**: `certificate` (TLS certificate), `json` (configuration handoff), `logs`
- **Permissions**: Storage Blob Data Contributor assigned to the Neologik Admin User Group at resource group level

10. Managed Identities

Creates two User Assigned Managed Identities with retry logic for replication delays:

1. **`neologik-script-runner-service-connection-<short-name>`**
    - Display Name: `Neologik Script Runner Service Connection - <short-name>`
    - Contributor (Subscription); Key Vault Secrets Officer and Azure Bot Service Contributor Role (Resource Group)
2. **`neologik-sql-managed-identity-<short-name>`**
    - Display Name: `Neologik SQL Managed Identity - <short-name>`
    - Directory Readers (Entra ID directory role)

11. Role Assignments

- Adds Neologik users and the signed-in user to all security groups (with graceful handling of replication delays)
- Assigns the Neologik Admin User Group its resource-group-scoped roles: Contributor, Azure Kubernetes Service Cluster Admin Role, Key Vault Administrator, Storage Blob Data Owner, Storage

Table Data Contributor, Azure Kubernetes Service RBAC Cluster Admin, Cognitive Services Data Reader, Cognitive Services OpenAI Contributor, Search Index Data Reader, Azure AI Developer (no subscription-scoped roles)
- All managed identities receive their roles with retry logic for replication delays

12. Output Generation

Creates two files next to the script:

- **Configuration JSON**: `Neologik_<org><index>_Configuration_<timestamp>.json` (includes script version; excludes secret values)
- **Log File**: `NeologikOnboarding_<timestamp>.log`

In interactive runs the configuration JSON is also uploaded to the Storage Account's `json` container (skipped in non-interactive mode).

Output Files

Configuration File

The configuration file contains all the information Neologik needs for the deployment (secrets are NOT included). Key contents:

- Tenant, subscription, resource group, region and organization details
- `SolutionName` / `SolutionShortName` (the naming tokens, including any overrides)
- `SecurityGroups` - names and object IDs of the four groups
- `AppRegistration` - the deployment SP's name, client ID, role assignments, Graph app roles and whether it was created by the script or client-provided
- `NceAuthApp` - the Platform Sign-in app's client ID
- `KeyVault`, `StorageAccount` (with container names), `ManagedIdentities`, `RoleAssignments`, `InvitedGuestUsers`

Note: Client secrets are stored securely in Key Vault and NOT included in the JSON output.

Log File

The log file contains detailed execution information: timestamps for all operations, success/warning/error messages, and troubleshooting information.

Security Considerations

Client Secret Storage

AUTOMATED: client secrets are automatically stored in Azure Key Vault:

- **Key Vault Name**: `kv<short-name>` (or your override)
- **Secret Names**: `neologik-deployment-service-principle-secret` (deployment SP), plus the Platform Sign-in app secret
- **Access**: Available to Neologik Admin User Group members via RBAC

You do NOT need to manually copy or store the secrets.

Least Privilege

- All resource-write grants are resource-group-scoped. The deployment SP holds only read-only **Reader** and cost-configuration-only **Cost Management Contributor** at subscription scope.
- `Role Based Access Control Administrator` (RG scope) replaces the legacy subscription-scoped User Access Administrator - it grants role-assignment writes only, with no deny-assignment powers.
- Microsoft Graph permissions are scoped app roles (`Application.ReadWrite.OwnedBy` manages only apps the SP owns; the Agent ID roles are agent-identity-specific) - no tenant-wide directory roles such as Application Administrator are granted.

Certificate Storage

The Storage Account uses **Microsoft Entra ID authentication only**:

- Shared key access is **disabled**
- Access controlled via RBAC (Storage Blob Data Contributor)
- All operations require Entra ID authentication

Permissions Required

The script requires elevated permissions:

- **Administrator** access on local machine (for module installation)
- **Subscription Owner** role (for resource creation and RBAC)
- **Global Administrator** role (for Entra ID operations)

Best Practices

- Run the script from a secure, trusted machine
- Review all prompts before confirming
- Store output files securely
- Audit the log file for any issues
- All secrets are stored in Key Vault, not in local files

Troubleshooting

PowerShell Version Issues

Problem: Script fails with version error

Solution:

```
# Let script update automatically, or manually install:
winget install Microsoft.PowerShell
```

Module Installation Fails

Problem: Cannot install required modules

Solution:

```
# Set execution policy
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

# Manually install modules
Install-Module -Name Az -Scope CurrentUser -Force
Install-Module -Name Microsoft.Graph -Scope CurrentUser -Force
```

Authentication Fails

Problem: Cannot connect to Azure

Solution:

1. Ensure you have an active internet connection
2. Verify your Azure credentials
3. Check that your account has required permissions
4. Try clearing Azure credential cache:

   ```
   Clear-AzContext -Force
   ```

Permission Errors

Problem: "Insufficient privileges" or "Access denied"

Solution:

1. Verify you have **Subscription Owner** role
2. Verify you have **Global Administrator** role in Entra ID
3. Check role assignments in Azure Portal
4. Contact your Azure administrator if needed

Guest User Invitation Fails

Problem: Cannot invite external users

Solution:

1. Check External Collaboration Settings in Entra ID
2. Ensure guest invitations are allowed
3. Verify email addresses are correct
4. Check spam folder for invitation emails

Role Assignment Fails with "BadRequest"

Problem: Script shows "BadRequest" errors when assigning roles

Solution:

- **Automatic**: The script includes retry logic for newly created principals
- This is normal for newly created principals (Service Principals, Security Groups, Managed Identities)
- Entra ID needs time to replicate resources across regions
- The script will automatically retry and succeed after replication completes
- No action needed - just wait for the retries to complete

Re-authentication Needed

Problem: Need to switch Azure accounts during setup

Solution:

1. When prompted "Do you want to re-authenticate to Azure?", press **Y**
2. Sign in with the correct Azure account
3. Ensure the new account has required permissions

Resource Already Exists

Problem: Script reports resource already exists

Solution: The script is designed to handle existing resources gracefully. It will:

- Skip creation if resource exists
- Validate existing configuration
- Only create missing components

This is normal and allows for re-running the script safely.

Interrupted Run

Problem: The script stopped part-way (error, closed window, lost connection)

Solution: Run it again. Progress is checkpointed to `NeologikCheckpoint.json` after each step; the script offers to resume and skips completed steps. The checkpoint is deleted automatically on success.

Resource Provider Registration Timeout

Problem: "Resource provider registration failed" or timeout message

Solution:

1. Wait 5 minutes as instructed in the error message
2. Run the script again - it will resume and check registration status
3. The script automatically retries registration

Resource provider registration can sometimes take time in Azure.

What to Share with Neologik

After successful completion, share the following with Neologik:

1. Configuration File

Send the generated `Neologik_<org><index>_Configuration_<timestamp>.json` file to support@neologik.ai (in interactive runs it is also uploaded automatically to the Storage Account's `json` container).

2. Azure Access

The invited Neologik team members are automatically added to the appropriate security groups with the necessary permissions (Key Vault secret access, Storage Account access). If you skipped guest invitations, agree the access route with Neologik instead.

3. Post-Script Manual Steps

Complete these steps in the Azure Portal:

Upload TLS Certificate

1. Navigate to the Deployment Storage Account (default `stneodeploy<org><env><region><index>`)
2. Go to Containers, then `certificate`
3. Upload your TLS certificate (.pfx file)
4. Use **Microsoft Entra ID** authentication (not access key)

Store Certificate Password

1. Navigate to the Key Vault (default `kv<org><env><index>`)
2. Go to Secrets
3. Create new secret:
    - **Name**: `neologik-deployment-certificate-pfx-secret`
    - **Value**: Your certificate PFX password

4. Certificate Requirements

The TLS certificate (PFX) must contain:

- Server certificate
- Full certificate chain (intermediate CAs)
- Root CA certificate
- Private key

Post-Installation Steps

After running the script:

1. **Review** the configuration file
2. **Upload** TLS certificate to the Storage Account (see manual steps above)
3. **Store** certificate password in Key Vault (see manual steps above)
4. **Share** the configuration file with the Neologik team

5. **Verify** guest users received invitations (unless skipped)
6. **Keep** the log file for troubleshooting

All secrets are securely stored in Azure Key Vault - no manual secret management required.

Support

Neologik Support

For issues specific to the Neologik Platform:

- Contact: support@neologik.ai

Script Issues

For issues with this onboarding script:

- Check the log file first
- Review troubleshooting section
- Contact the Neologik team for assistance

Version History

See `VERSION_HISTORY.md` for the full change log.

License

Copyright Neologik. All rights reserved.

This script is provided for Neologik customer onboarding purposes only.

Contributing

This is a Neologik repository. For suggestions or improvements, contact the Neologik development team.

Neologik AI - Intelligent Solutions for Modern Enterprises